

WZIP and WLZ4: LZ77 Codecs with Match-Length-Dependent Sliding Windows

Yingquan (Cody) Wu

Tenafe Inc.

591 W. Hamilton Ave

Campbell, CA 95008, USA

icodywu@gmail.com

Abstract

An LZ77 codec bounds every match by one sliding window, trading cheap offsets for short matches against reach for long ones. Match-length-dependent windows give finer control over this trade-off: each match length has its own window, and the decoder infers the offset range from the decoded length. We present two such codecs: WLZ4, a byte-aligned LZ4-class format with one-byte and flagged offsets, and WZIP, an entropy-coded Zstandard-class format. Each compressed block begins with its decoded length, from which WZIP derives its widest window and a decoder sizes its output. LZ4 blocks may expand incompressible input by 0.4%, whereas WLZ4 adds at most 15 bytes and WZIP 2. We describe the formats, threaded match finders, decoders and optimal parser. On Silesia, WLZ4 reaches a ratio up to 16% above LZ4HC's, decoding more slowly, and WZIP exceeds Zstandard 22's ratio while decoding at 86% of its speed (on enwik9, as fast).

1 Introduction

LZ77 [1] replaces a repeated substring by a pair (ℓ, d) : copy ℓ bytes from d bytes back. Every practical LZ77 codec bounds d by a window W , and the choice of W is a compromise: a wide window exposes distant repetitions but makes every offset expensive, so that a 3-byte match within a 64 KB window saves little or nothing, while a narrow window misses long repetitions further back. Byte-aligned codecs such as LZ4 [2] fix the window at 64 KB and the minimum match at 4 bytes; entropy-coded codecs such as DEFLATE [3], Zstandard [4], Brotli [5] and LZMA/xz [6, 7] ease the compromise with variable-length offset codes, and LZMA and Brotli condition them on the match length (2, 3, 4, 5+), which changes the offsets' probabilities but not their range: every length still reaches the whole window.

The top of each class is hard to move. Zstandard at level 22 combines a 128 MB window, a binary-tree optimal parser and FSE entropy coding [8], refined over a decade (its `--max` mode, new in version 1.5.7, compresses enwik9 5.6% more at 12 times the time and 6 times the memory [9]); the codecs we measured that compress Silesia further, Brotli at level 11 and xz at `-9e`, rely on context-modeled literal coding and decode 2.6 and 8.8 times slower. LZ4HC at level 12 is similarly the strongest byte-aligned parser at LZ4's decoding speed. We aim to improve on both, keeping each class's decoding structure: WZIP, with Huffman coding only, exceeds the ratio of Zstandard 22 by 0.8–1.1% on Silesia at levels 11–12 while decoding at 86% of its speed (by 1.3% at level 13, decoding 19% slower), and at level 13 by 1.8% on the

1 GB enwik9, also above xz, decoding about as fast; WLZ4 reaches ratios up to 16% above LZ4HC 12’s on Silesia and 29% on enwik9, decoding at most 33% slower.

Match-length-dependent windows assign each length ℓ a window $W(\ell)$, nondecreasing in ℓ . Short matches are admitted only at short distances, where their offsets are cheap, and long matches reach the whole history; since lengths are decoded before offsets, the decoder knows $W(\ell)$. Byte-aligned formats have tied offset fields to lengths before: Snappy [10] limits its one-byte-offset copies (11-bit offsets) to lengths 4–11, and Lizard [11] gives 16-bit offsets to matches of 4 bytes or more and 24-bit offsets only to matches of 16 or more. WLZ4 instead gives every match longer than 3 bytes a flag that buys one more offset byte, so that the 8 MB window opens at length 6 rather than being reserved to long matches. Our earlier abstract [12] studied a nondecreasing schedule over all lengths with an LZ4-based prototype, and [13] analyzes it: exact greedy parsing under length-dependent windows remains universal when the full window starts at a length $o(\log W)$, and a calibrated schedule never raises the minimum token cost over a single window. This paper is about engineering: two complete codecs, the design decisions the scheme enables, and measurements against current codecs. New relative to [12] are the flagged offset, WZIP’s size-derived schedule and joint symbols, the optimal parser, threads per index, and the evaluation. Sections 2–4 describe the scheme and both formats, Section 5 their parser, and Section 6 compares them with LZ4 1.10, Zstandard 1.5.6, Brotli 1.1.0 and xz 5.6.2.

2 Length-dependent windows

A match (ℓ, d) at input position i copies $x_{i-d}, \dots, x_{i-d+\ell-1}$ to $x_i, \dots, x_{i+\ell-1}$, possibly overlapping itself. Under a schedule $W(\ell)$, (ℓ, d) is admissible if $d \leq W(\ell)$; a constant W is the single-window case. The schedule binds matches with a new offset: a repeat offset, coded by its cache slot, and WZIP’s run token are not restricted. In a byte-aligned format the schedule appears as offset fields of different sizes, selected in WLZ4 by the length code and a flag, in an entropy-coded format as offset alphabets that grow with the length; either way the decoder needs no extra memory. Both codecs start at length 3: a length-2 match costs WLZ4 as much as the two literals it replaces, and within 16 bytes it lowered WZIP’s ratio on 4–8 KB blocks by 0.01–0.08%.

The input length in the stream. A schedule may also depend on the input length n : a window wider than the input is wasted, and one that covers it lets the longest matches reach everything. Both formats therefore begin with n (two bytes below 32 KB, four otherwise; a stored WZIP input begins with a zero instead). The size costs at most four bytes and serves three purposes: WZIP derives a size-adapted base schedule $W_n(\ell)$ from it; the decoder allocates its output before decoding; and decoding can be checked against a known end. Snappy also stores the size up front [10]; the LZ4 block format omits it, LZ4 and Zstandard frames record it optionally, xz in its index, gzip modulo 2^{32} in a trailer, and Brotli not at all.

The schedule also shapes the search [12]: each class of lengths gets its own index, keyed by a hash of its minimum length and covering only the class’s window. Indexes on short seeds stay short even in repetitive data, long matches come from an index on long seeds over the whole window, and each index searched nearest first yields the

Table 1: WLZ4 match codes (low nibble of the token) and offset fields; offset 0 ends a block.

Code	Length	Offset field	Window
0	3	1 byte	256 B
1, 2	4, 5	flagged: 1 byte (flag 0) or 2 bytes (flag 1)	128 B or 32 KB
3–14, 15	6–17, 18+ext.	flagged: 2 or 3 bytes	32 KB or 8 MB

nearest offset for each length, the cheapest when offset fields have fixed widths.

3 WLZ4: a byte-aligned multi-window format

Format. A block starts with its decoded size (2 bytes below 32 KB, else 4), so that, unlike an LZ4 block, it needs no size from the application. It holds sequences, each a token, a literal run and a match, as in LZ4. The token’s high nibble is the literal-run length; its low nibble is a match code c for length $3 + c$ (Table 1), code 15 for 18 plus an extension. Code 0 takes a one-byte offset; every other code takes a *flagged offset*: a little-endian field of $1 + [c > 2]$ bytes, or one more if its low bit is set, the offset filling the remaining bits. A nibble value of 15 announces an extension e : one byte if $e < 252$, else $251 + m$ followed by $e - 252$ in m bytes. A zero offset ends the block.

The flag is what makes the far window pay: an earlier variant spent match code 15 on far matches, which then always needed a length extension, and compressed Silesia 2.6–2.8% worse (Section 6). The extension code keeps the size of an extension logarithmic, and every encoder writes a sequence only if the output stays within four bytes of the input it covers, so incompressible input expands by at most 15 bytes, including the header, where an LZ4 block may add $n/255$, or 0.4%.

Decoding. The decoder reads four bytes v at the offset position; with $f_c = [c > 0]$ and $b_c = [c > 2]$ from two 16-entry tables, the offset field has $1 + b_c + (v \wedge f_c)$ bytes and the offset is $v \gg f_c$, masked: one rule for every code, without a branch. Because the flag makes the position of the next token depend on a byte just read, the decoder takes the next token from v unless an extension follows, keeping a second load off each sequence’s critical path. Matches of up to 17 bytes at offsets of 8 or more take 8-byte copies, longer ones 32-byte copies, and offsets below 8 pattern copies, as LZ4’s.

Encoders. The fast and lazy levels probe two hash tables (3- and 5-byte hashes) and take a candidate of at least 3 bytes within 256 B, 5 bytes within 32 KB, or 6 bytes within 8 MB. The hash-chain levels 0–7 walk a 5-byte chain over 64 KB, compare candidates by net gain (length less the offset bytes beyond two), and probe two far positions: the uncapped previous position of the same 5-byte hash, and the previous position of the same 8 bytes from a table sized to the input (at most 2^{20} entries; it alone moved level 6 on Silesia from 2.70 to 2.81). Levels 8–12 run the optimal parser (Section 5) over 3-, 4- and 6-byte chains (256 B, 64 KB, 8 MB). Each level reaches at most its own far window: 8 MB at levels 7 and 12, half per level below.

4 WZIP: an entropy-coded multi-window format

Format. WZIP codes an input of 32 KB or more as one block; smaller inputs (7 of the 29 small files) go to WZIP_M, a variant with fixed 8 and 32 KB windows and two interleaved sequence streams, and the repository adds WZIP_S for 4–8 KB storage pages. It starts with the decoded size n , which fixes the base schedule and its widest window, not a compression parameter as Brotli’s window bits or xz’s dictionary size are; a three-byte header then gives the widths the encoder chose for lengths 3–7, as gaps below the widest. The schedule has entries for lengths 3, 4, 5, 6, 7 and 8+: the power of two of n selects the base widths of the short lengths, and the widest window is raised to cover the whole input, up to 2^{27} bytes for lengths 8+ and 2^{26} for shorter ones. The longest matches thus reach every earlier byte of any input up to 128 MB at no cost in decoder memory, since a one-shot decoder holds the whole output anyway, while small inputs keep small windows and offset alphabets. For a 16 MB input the base widths are 12, 16, 20, 22, 25 and 25 bits, and the optimal levels, whose parser prices each offset by its code length, widen lengths 3–6 to 18, 22, 24 and 24. Literals are Huffman-coded [14] in their own stream; sequences, in blocks of 2^{14} , are coded as (i) a *joint symbol* of the offset-cache slot (four recent offsets or a new one), the literal-run class (0, 1, or 2+ literals) and the length symbol (lengths 3–4095 with extra bits, and a run token), 1020 values in all, or a 204-value alphabet without the slot, chosen per block; (ii) a literal-run symbol for runs of two or more; and (iii) for a new offset, a symbol from the Huffman table of the length’s group (lengths 3, 4, 5 and 6+, or eight finer groups at the highest level), plus raw bits. The cache is kept in move-to-front order. The run token codes a run of the preceding byte with its count in the offset field, bypassing the cache. The joint symbol captures the dependence of slot, class and length that separate Huffman codes, with their one-bit floor, miss. An input WZIP cannot shrink by 32 bytes is stored after a zero size field (2 bytes more).

Encoders and decoder. Level 0 probes one hash table on 6-byte seeds in a 1 MB window; levels 1–3 are greedy and 4–6 lazy, comparing candidates by an estimated gain (four bits per byte less $\log_2 d$). Levels 7–13 run the optimal parser with three indexes: a 3-byte chain for lengths 3–4 searched to $W(4)$, a 5-byte chain for lengths 5–6 searched to $W(6)$, and a binary tree on 7-byte seeds over the whole window; each chain stops at a match long enough for the next index or after 16 steps. Each level searches at most its own window: 2^{27} bytes at levels 6 and 13, half per level below, narrower windows staying below it; streams keep the windows of their size, so decoders need no level. Where matches often reach 1 MB back or more, beyond the L2 cache, the decoder runs the next block of sequences as a pipeline, like Zstandard’s long-offset decoder: each step decodes a sequence, prefetches its source and executes the one decoded 16 steps before (enwik9: 62% faster).

Threads. The indexes depend on the input alone, so each runs ahead of the parser in a thread of its own (the tree split by hash bucket), with the output of one thread: level 11 compresses Silesia 3.0 times as fast with 6 threads on one core complex. More threads compress 64 MB blocks at once, each with the level’s window before it as a dictionary: enwik9 at level 11, 17 times as fast with 105 threads, 0.007% larger.

Parse a segment from position p , with carried literal run r_0 and offset cache R_0 . A node (c, s) is the cheapest known path from p to $p + c$ in state s ; it stores price, last step, previous state, pending literal run and (WZIP) offset cache. A pending run r is in state $\sigma(r) = 0$, or (WZIP levels 12–13) in its literal-run class $\sigma(r) = \min(r, 2)$. Initialize $(0, \sigma(r_0))$. For $c = 0, 1, \dots$:

1. if $c > 0$, for each state s at $c - 1$, with pending run r , relax $(c, \sigma(r + 1))$ by literal x_{p+c-1} and the change in the literal-run code;
2. stop at the last position reached (if $c > 0$), segment limit 4096 or input end;
3. gather candidates at $p + c$: per-class matches, and (WZIP) each state’s cached offsets and a run;
4. for each state s , candidate (L, d) and length $\ell \leq L$ that d may code, relax $(c + \ell, 0)$ with $\text{price}(c, s) + \text{cost}(\ell, d \mid s)$; (WZIP) its cache is the predecessor’s with d moved to the front, unchanged for a run;
5. on a candidate of the sufficient length, take it from the cheapest state and stop.

Back-trace from the cheapest state at the end, emit, and continue from there.

Figure 1: The optimal parser of WLZ4 (levels 8–12) and WZIP (levels 7–13).

5 Optimal parsing with length-dependent windows

Both codecs use the parser of Figure 1, an approximate shortest path over input segments like the “optimal” parsers of LZ4HC and Zstandard. Four aspects differ.

Candidates. A single-window parser can price every length up to the longest match with the nearest candidate reaching it, the cheapest if offset prices do not decrease with distance. Under a schedule this fails twice. First, a candidate (L, d) codes only the lengths $\ell \leq L$ with $d \leq W(\ell)$: a far candidate may not code its own prefixes. WZIP sorts the matches of its three indexes by offset, keeps each only if it is longer than every nearer one and admissible at its own length, and prices each length with the nearest such candidate that admits it, skipping lengths no admissible candidate reaches. Second, in a byte format the offset cost steps with the distance, so WLZ4 keeps one candidate per offset class: the longest match within 128 B and within 256 B, within 32 KB and within 64 KB (each pair from one nearest-first chain walk that snapshots its best when crossing the inner bound), and beyond 64 KB, pricing each length with the cheapest that codes it. Cached offsets are coded by slot, not bound by the schedule, so WZIP also tries each cached offset at every position with every length, and a run, even of two bytes, as a run token.

Prices. WLZ4 prices are exact bytes; each node’s pending literal run is charged when the run ends. WZIP prices symbols in 1/256 bit from smoothed running frequencies, refreshed every 32 KB: a match costs its joint symbol, extra bits, and for a new offset its group’s offset symbol and raw bits; the literal-run code is charged incrementally. As the parser’s output feeds its prices, the initial counts matter: a

steep prior over lengths makes the parser split long matches, and the resulting counts keep short lengths cheap; WZIP starts from a weak prior halving every eight.

States. The joint symbol codes the literal-run class, so a match’s cost depends on how its path arrived: after a match, one literal, or more. Keeping one path per position commits to one class and misprices the others. From level 12 WZIP keeps three nodes per position, one per class (a literal moves class s to $\min(s+1, 2)$, a match to 0), each with its own offset cache. This remains a bounded dynamic program, not a bit-optimal parse: paths that reach the same position and class with different caches or pending runs are merged, and segments, candidate limits and estimated prices restrict it further, as in the other “optimal” parsers of practical codecs.

Passes. Running prices lag behind changes in the data. At level 13 a first pass (shallow, one state) records the symbol counts of every 128 KB region, and the second pass prices each region from those counts: prices anticipate the local statistics that the second parse largely reproduces, one step of alternating between parse and code.

Relation to existing parsers. LZ4HC prices every length up to one longest match in exact bytes; Zstandard and LZMA [6] keep one path per position with its coder state, Zstandard seeding its strongest level with a trial pass over the first block; bit-optimal parsing [15] is exact for fixed costs. New here are per-class candidates with per-length admissibility, a state per literal-run class and per-region first-pass prices.

6 Experiments

Setup. Nodes of two AMD EPYC 9334 (Zen 4, 32 cores each, clock capped at 2.7 GHz, 32 MB of L3 per 8-core complex), Ubuntu 22.04. Each run is one thread pinned to a core complex of its own, and every configuration ran twice, on identical nodes, the same day. WLZ4, WZIP, LZ4 1.10.0, Zstandard 1.5.6 (with its assembly Huffman decoder), Brotli 1.1.0 (window 2^{24} , its standard maximum) and xz 5.6.2 (no check) were built from source with GCC 14.2 -O2. Each file is compressed whole, by one library call. Ratio is total input over total output; speeds are total input over total time after a one-second warm-up, best of 3 compressions for fast codecs (1 otherwise) and of 20 decompressions, over both runs; every file is verified. LZ4 decodes with its bounds-checked `LZ4_decompress_safe`; WLZ4 and WZIP decode in their opt-in trusted mode, needing no size from the caller as each stream stores it (their default, checked mode decodes Silesia at most 9% slower). Encoder memory is the growth of the peak resident set during compression, not counting contexts created beforehand. Corpora: Silesia [16] (12 files, 5–51 MB, 211.9 MB); as small files, Canterbury and Calgary [17, 18] (29 files, 3.7 KB–1 MB, 6.0 MB); and as large inputs enwik8 and enwik9 [19], the first 10^8 and 10^9 bytes of an English Wikipedia dump.

The LZ4 class (Table 2, Figure 2). On Silesia every WLZ4 level from 6 up reaches a higher ratio than LZ4HC at its highest level: level 6 reaches 2.820 against 2.743 at 3.0 times LZ4HC-12’s compression speed, and the optimal levels 8, 10 and 12 reach 2.812, 3.009 and 3.186, up to 16.1% above it; narrower far windows cost the lower levels (level 2: 2.614). The fast and lazy levels gain 13–16% over LZ4’s default mode at 42–45% of its speed. On the small files the far window has little to find, and the 32 KB two-byte window leaves the hash-chain levels about 2% below LZ4HC at

Table 2: The LZ4 class: ratio, and compression and decompression speed (MB/s), on Silesia and on Canterbury+Calgary; encoder memory (MB) on Silesia.

Codec, level	Silesia			Canterbury+Calgary			Memory (MB)
	Ratio	Comp.	Dec.	Ratio	Comp.	Dec.	
LZ4 (default)	2.101	731	4718	2.074	626	4656	0.0
LZ4HC 9	2.721	38.8	4458	2.751	31.7	4559	0.6
LZ4HC 12	2.743	13.4	4568	2.771	4.87	4437	0.9
WLZ4 fast	2.373	326	3057	2.333	289	2855	0.6
WLZ4 lazy	2.432	306	3262	2.403	274	3005	0.5
WLZ4 2	2.614	70.5	3479	2.627	69.4	3213	1.7
WLZ4 6	2.820	40.1	3520	2.697	34.9	3446	5.3
WLZ4 8	2.812	20.4	3331	2.773	17.5	3068	7.5
WLZ4 10	3.009	9.23	3461	2.820	10.7	3123	15
WLZ4 12	3.186	1.71	3504	2.851	2.81	3177	41

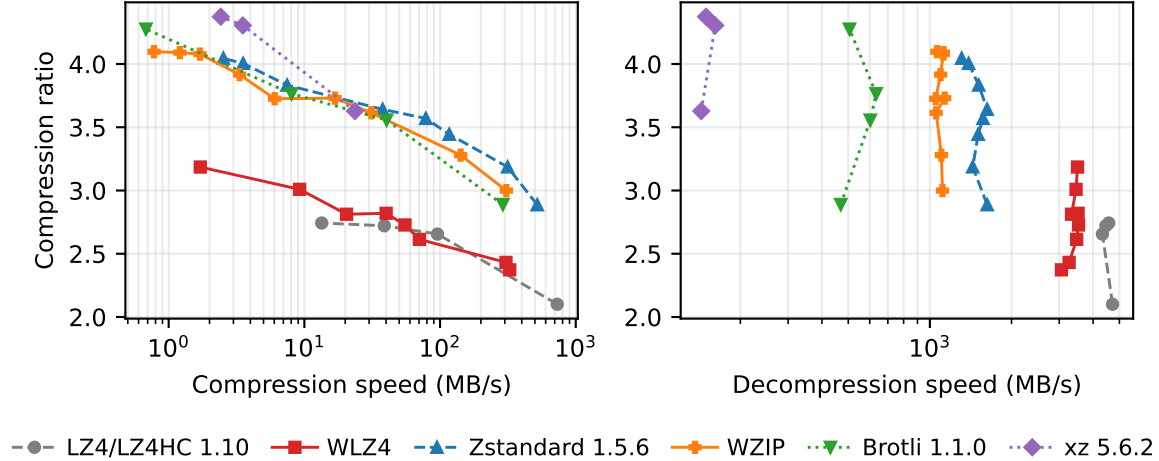


Figure 2: Silesia: ratio against compression and decompression speed (log scale).

similar speed, while levels 10 and 12 still exceed LZ4HC-12 (2.820 and 2.851, against 2.771). The price is decoding speed: 2.9–3.5 GB/s, against 4.4–4.7 GB/s for LZ4. Against Zstandard the trade-off reverses: level 12 matches Zstandard 3’s ratio (3.186, Table 3) and decodes 2.4 times as fast, but compresses 180 times more slowly.

The Zstandard class (Table 3). WZIP’s levels 0–9 lie close to Zstandard’s trade-off curve on Silesia but slightly below it (level 9: 3.916 at 3.3 MB/s; Zstandard 19: 4.005 at 3.5 MB/s). Its top levels exceed Zstandard 22’s ratio: level 11 reaches 4.078, 0.8% above Zstandard 22’s 4.045, at 67% of its compression speed, and level 13 reaches 4.097 (+1.3%); on the small files level 13 exceeds Zstandard 22 by 0.8% (Zstandard 1.5.7 changes these ratios by at most 0.06%). WZIP decodes 14–32% slower than the Zstandard level of similar ratio in Table 3 (least at the top: levels 11 and 12 decode

Table 3: The Zstandard class: ratio, and compression and decompression speed (MB/s), on Silesia and on Canterbury+Calgary; encoder memory (MB) on Silesia.

Codec, level	Silesia			Canterbury+Calgary			Memory (MB)
	Ratio	Comp.	Dec.	Ratio	Comp.	Dec.	
Zstandard 1	2.887	520	1627	3.276	420	1554	0.9
Zstandard 3	3.186	315	1439	3.572	269	1320	1.7
Zstandard 19	4.005	3.54	1389	4.209	4.72	1154	82
Zstandard 22	4.045	2.53	1310	4.210	3.54	1154	642
WZIP 0	2.999	308	1113	3.378	228	964	15
WZIP 3	3.614	31.2	1055	3.753	52.4	887	154
WZIP 9	3.916	3.33	1095	4.147	5.99	873	183
WZIP 11	4.078	1.70	1121	4.186	3.47	884	566
WZIP 12	4.090	1.21	1122	4.195	2.14	886	948
WZIP 13	4.097	0.78	1062	4.243	1.49	820	952
Brotli 11	4.276	0.68	505	4.490	0.80	471	241
xz -6	4.305	3.52	162	4.392	5.19	154	94
xz -9e	4.374	2.42	149	4.396	2.49	144	505

at 1121 and 1122 MB/s against Zstandard 22’s 1310). Part of the gap may come from WZIP’s Huffman tables, several per block, against Zstandard’s FSE [4, 8]. WZIP decodes 2.2 times as fast as Brotli 11 and 7.5 times as fast as xz, which compress Silesia further (4.276 and 4.374) at compression speeds of 0.7 and 2.4 MB/s. Memory: level 11 (32 MB window) 566 MB, Zstandard 22 642 MB, levels 12–13 950 MB.

Table 4: Large inputs: ratio, and compression and decompression speed (MB/s), on enwik8 (100 MB) and on enwik9 (1 GB); encoder memory (MB) on enwik9.

Codec, level	enwik8			enwik9			Memory (MB)
	Ratio	Comp.	Dec.	Ratio	Comp.	Dec.	
LZ4 (default)	1.746	540	4354	1.964	592	4457	0.0
LZ4HC 12	2.386	18.2	3679	2.691	20.4	3912	0.8
WLZ4 6	2.523	35.7	2982	2.859	39.7	3237	5.2
WLZ4 10	2.766	5.80	2871	3.131	6.89	3100	15
WLZ4 12	3.046	0.88	2662	3.470	1.07	2858	41
Zstandard 19	3.712	2.50	1233	4.250	2.58	1358	81
Zstandard 22	3.957	1.53	971	4.676	1.41	966	649
WZIP 11	3.900	1.18	1037	4.505	1.23	1115	745
WZIP 12	3.972	0.93	1001	4.644	0.90	1056	1256
WZIP 13	3.983	0.50	939	4.759	0.47	958	2102
Brotli 11	3.885	0.61	532	4.477	0.62	594	205
xz -9e	4.027	1.45	144	4.722	1.42	166	674

Large inputs (Table 4). Our settings were tuned on Silesia and the small files,

and WZIP’s windows for inputs above 64 MB on enwik8, the first tenth of enwik9, so enwik9 is not an independent test. WLZ4 12’s ratio is 28–29% above LZ4HC 12’s (level 10: 16%), but it decodes 27–28% slower, and level 6 exceeds LZ4HC 12 by 6% at twice its compression speed. Here the level’s window decides: WZIP 12 (64 MB) exceeds Zstandard 22 by 0.4% on enwik8 but falls 0.7% short on enwik9, and WZIP 13 (128 MB) exceeds it by 0.7% and 1.8%; on enwik9 WZIP 13 compresses best of all codecs tested (xz $-9e$: 4.722) and with its pipelined decoder decodes within 1% of Zstandard 22’s speed, but its encoder uses 2.1 GB (Zstandard 22: 0.65 GB).

Ablation. Table 5 compares byte-aligned formats under optimal parsing (ours, full windows; LZ4HC-12’s for LZ4). On Silesia the far window adds 10–12% over two windows, and moving it from a match code to the flag adds 2.6–2.8% more, since far matches then need no length extension (the flag also halves both windows); on the small files, the flagged format’s 32 KB two-byte window costs 0.2–0.3%, and flagging lengths 4–5 too adds 0.2% on both corpora. Assigning offset sizes by length alone (e.g., 3 bytes from length 6) lost 1–3% against the far code on Silesia and 11% on the small files, where 47–64% of matches have length 6+ within 64 KB.

WZIP’s parts, at level 11 with the full window on Silesia (4.080): coding the cache slot with the offset, not in the joint symbol (same parse), lowers the ratio by 0.28%, and one offset table for all lengths instead of one per length group (reparsed) by 0.58%. Giving lengths 3–7 the widest window, with five offset tables in both runs, changes the ratio by +0.002% but on mozilla raises peak memory from 1.06 to 1.44 GB and compression time by 11% (laptop): here the gain lies in coding offsets per length, and the windows keep the indexes of short seeds small. Three parser states add 0.25% (level 12), the second pass with eight offset groups 0.17% (level 13).

Table 5: Byte-aligned formats under optimal parsing: ratio at levels 10 and 12.

Offset fields	Silesia		Cant.+Calg.	
	L10	L12	L10	L12
LZ4: 2 B, 64 KB (LZ4HC-12’s parser)	—	2.743	—	2.771
1 B 256 B (len. 3–4); 2 B 64 KB	2.749	2.769	2.772	2.789
+ far code: 3 B 16 MB, length extension	3.015	3.091	2.822	2.854
flag instead: 2 B 32 KB or 3 B 8 MB (len. 4+)	3.095	3.179	2.815	2.846
WLZ4: len. 4–5 flagged 1 B 128 B or 2 B 32 KB	3.101	3.186	2.820	2.851

7 Discussion and conclusion

Limitations. WLZ4 decodes 23–33% slower than LZ4HC 12 on Silesia, and up to 28% on enwik8 and enwik9, likely for three reasons: its parses contain more and shorter matches; the flag adds a data-dependent load to each sequence; and on large inputs a third of its matches copy from 64 KB to 8 MB back, beyond L2. A parse that also prices decoding time could trade ratio for speed.

Conclusion. Match-length-dependent windows let an LZ77 format give short matches cheap offsets without denying long matches a large history: as offset fields and a flag

in a byte-aligned format, where the windows themselves raise the ratio, and as offset alphabets coded per length group in an entropy-coded one, where under our optimal parser the gain comes from the per-length coding and the windows keep the search fast and small. Recording the input length up front costs at most four bytes and lets the schedule itself adapt to the input. Source code, the benchmark harness and scripts reproducing the tables are available at <https://github.com/icodywu/WZIP>.

References

- [1] J. Ziv and A. Lempel, “A universal algorithm for sequential data compression,” *IEEE Trans. Inf. Theory*, vol. 23, no. 3, pp. 337–343, 1977.
- [2] Y. Collet et al., “LZ4: extremely fast compression,” version 1.10.0, 2024. <https://github.com/lz4/lz4>.
- [3] P. Deutsch, “DEFLATE compressed data format specification version 1.3,” RFC 1951, 1996.
- [4] Y. Collet and M. Kucherauw, “Zstandard compression and the ‘application/zstd’ media type,” RFC 8878, 2021.
- [5] J. Alakuijala et al., “Brotli: A general-purpose data compressor,” *ACM Trans. Inf. Syst.*, vol. 37, no. 1, art. 4, 2019.
- [6] I. Pavlov, “LZMA specification (draft version),” 2015. <https://www.7-zip.org/sdk.html>.
- [7] L. Collin et al., “XZ Utils,” version 5.6.2, 2024. <https://tukaani.org/xz/>.
- [8] J. Duda, “Asymmetric numeral systems: entropy coding combining speed of Huffman coding with compression rate of arithmetic coding,” arXiv:1311.2540, 2013.
- [9] Zstandard v1.5.7 release notes, 2025. <https://github.com/facebook/zstd/releases/tag/v1.5.7>.
- [10] Google, “Snappy compressed format description,” 2011. https://github.com/google/snappy/blob/main/format_description.txt.
- [11] P. Skibiński, “Lizard v1.x block format,” 2016. <https://github.com/inikep/lizard>.
- [12] Y. Wu, “Improved LZ77 compression,” in *Proc. Data Compression Conf.*, 2021, p. 377.
- [13] Y. Wu, “Improved LZ77 compression with match-length-dependent sliding windows,” submitted to *IEEE Trans. Inf. Theory*, 2026. arXiv:2610.06530.
- [14] D. A. Huffman, “A method for the construction of minimum-redundancy codes,” *Proc. IRE*, vol. 40, no. 9, pp. 1098–1101, 1952.
- [15] P. Ferragina, I. Nitto and R. Venturini, “On the bit-complexity of Lempel-Ziv compression,” *SIAM J. Comput.*, vol. 42, no. 4, pp. 1521–1541, 2013.
- [16] S. Deorowicz, “Universal lossless data compression algorithms,” Ph.D. dissertation, Silesian University of Technology, 2003.
- [17] R. Arnold and T. Bell, “A corpus for the evaluation of lossless compression algorithms,” in *Proc. Data Compression Conf.*, 1997, pp. 201–210.
- [18] T. C. Bell, J. G. Cleary and I. H. Witten, *Text Compression*. Prentice Hall, 1990.
- [19] M. Mahoney, “Large text compression benchmark,” 2006. <http://mattmahoney.net/dc/text.html>.